Installation & Setup For Windows

For Windows :

Step 1: Install VS Code(Code Editor)

- Go to: https://code.visualstudio.com/
- While installing: tick these options:
  - "Add to PATH"
  - "Open with Code"

Step 2: Install C Extension

- Go to Extensions (left sidebar or Ctrl + Shift + X)
- C/C++ (Microsoft) → install it
- code runner

Step 3: Install a C Compiler (MinGW)

- Go here: https://sourceforge.net/projects/mingw/
- Click Download.
- Select Packages : In the manager
- Right-click and mark these:
  - mingw32-gcc-g++
  - mingw32-base
  - Click Installation → Apply Changes

Step 4: Set Environment Variable (VERY IMPORTANT)

If you skip this, your compiler won't work.

- Search "Environment Variables" in Windows
- Click Edit the system environment variables
- Click Environment Variables
- Under System Variables, find Path

- Click Edit → New
- Add this path:
  - C:\MinGW\bin
- Click OK everywhere

Step 5: Verify Installation

- Open Command Prompt and type:
  - gcc --version
- If you see version info → DONE
- If not → you messed up the PATH. Fix it.

Step 5: Write and Run a C Program

- Create file - hello.c
- Compile via console :
  - gcc hello.c -o hello.exe
  - hello.exe

Installation & Setup For Mac

For Mac :

Step 1: Install VS Code(Code Editor)

- Go to: https://code.visualstudio.com/
- Download for macOS (Apple Silicon works fine)
- Open .zip
- Drag Visual Studio Code into Applications

Step 2: Install Compiler

- Download Xcode from App Store
- Through CMD :
  - Run this command : xcode-select --install
  - Verify C is working : clang --version

Step 3: Install VS Code Extensions

- Open VS Code → Extensions (left sidebar)
- Install :
  - C/C++ (by Microsoft) → REQUIRED
  - Code Runner

Step 4: Write and Run a C Program

- Create file - hello.c
- Compile via console :
  - clang hello.c -o hello
  - ./hello

BASICS

- Just like we use Hindi/English to talk to each other...
- We need a language to talk to computers.
- That language is called a **programming language.**
- Examples:
- C, C++, Java, Python, Ruby
- Fun fact: There are **8000+** programming languages in the world!

What is C?

- C is a programming language used to give instructions to the computer.
- Created by Dennis Ritchie in the 1970s at Bell Labs.
- It's known for being:
  - Fast
  - Efficient
- Used to make operating systems, games, software, and more.

Can Computers Understand C?

- No! Computers do not understand C or any human language directly.
- Computers only understand Binary Language — 0s and 1s.
  - 1 means current is passing ✅
  - 0 means no current ❌

Compiler

- A tool that translates your C code into Machine Code (Binary).
- Without a compiler, the computer won't understand your code.

What is HLL?

- We write programs in HLL(High Level Language) because it is easy for humans to read and write.
- Languages like C, C++, Java, Python are called High-Level Languages.

What is LLL?

- LLL is close to machine language (like binary 0s and 1s).
- It is difficult for humans but easy for the computer.

What is Code?

Code is a **set of instructions** written by a person (called a programmer) to tell the computer what to do.

What is Program?

A program is a **collection of many lines of code** that work together to do a task.

Memory Types

Primary Memory

- Directly accessible by the CPU.
- Example: RAM (Random Access Memory) and Cache Memory
- Like a rough notebook — temporary and fast.

Secondary Memory

- Used for permanent storage.
- Example: Hard Disk, SSD, Pen Drive
- Like your final exam notes — saved permanently.

RAM, CPU & Cache Memory

RAM(Temporary Memory)

- Stores active apps/data while system is running.
- Data clears on shutdown.
- Like opening Instagram or YouTube — they load into RAM.

CPU(Central Processing Unit)

- Brain of the computer — performs all calculations & tasks.
- Works with RAM to process instructions.

Cache Memory

- Super-fast, very small memory inside/near the CPU.
- Stores frequently used data.
- Like keeping a pen in your hand — for quick access.

Operating System (OS)

- A system software that manages all hardware and software.
- Examples: Windows, Linux, macOS, Android.
- It acts as a bridge between user and computer hardware.

Files & Folders

- File = A single document (text, image, video, etc.)
- Folder = A container to organize multiple files.
- Like a library: Folders = shelves, Files = books.

Terminal / CMD

- A text-based interface to interact with the computer.
- Used to run commands directly.
- OS examples:
  - Windows: CMD (Command Prompt)
  - macOS/Linux: Terminal

Common Terminal Commands

Command	Description
cd foldername	Change directory (move into a folder)
mkdir foldername	Make/create a new folder
ls / dir	List files/folders in the current folder
del file.txt	Delete a file
cls/clear	Clear the screen

Environmental Variables

- Store system-level settings like:
  - Where programs are installed
  - Default paths, user info, etc.
- Example: After installing a C compiler, we add its path to environment variables so the system can find and run it from anywhere.

Comments & Variables

Comments

- Comments are notes or explanations you write in your code.
- The computer ignores them. They're only for humans to understand the code better.

Types of C

- Single Line Comment - // This is a comment
- Multi-line comment -

Syntax

- Just like English has grammar rules, C has coding rules.
- Syntax means the rules of the language.
- If you break the syntax, you get an error.

Keywords vs Words

- Special reserved words in C, that have special meaning and cannot be used as variable names are called Keywords.
- auto, break, case, char, const, continue, default, do, double, else, enum extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static struct, switch, typedef, union, unsigned, void, volatile, while

Variables

- A variable is a container that stores data like numbers, characters, etc.
- This value can be changed during the execution of the program.
- Before use, you need to declare and define it.

Variable Declaration

- int age;

Variable Initialisation

- age = 18;

Variable Declaration & Initialisation

- int age = 18;

Rules for Naming Variables

- Must start with an alphabet or underscore _
- Can end with digit.
- Special characters allowed: _ and $ only.
- Cannot use keywords.
- C is case-sensitive.

Code Readability

When you name your variables or functions, follow a good style.
Use either of these two:

- camelCase - First word small, then every next word starts with Capital letter.
  e.g : studentName
- snake_case - All words are in small letters, and you use _ (underscore) to separate them.
  e.g : student_name;

Data Types

C is a statically typed language

- You must declare the type of every variable.
- Once declared, the type cannot be changed.
- This is because each type uses different memory and supports specific operations.

Types of Data

1. Primitive Data Types
   - The most basic data types that are used for representing simple values.
   - int, char, float, double, void
2. Derived Data Types
   - Derived from the primitive or built-in datatypes.
   - array, pointers, function
3. User Defined Data Types
   - Defined by the user himself.
   - structure, union, enum

Format Specifier

Special token that begin with % symbol, followed by a character that specifies the data type.

Format Specifier	Description
%c	For character type.
%d, %i	For signed integer type.
%f	For float type.
%ld or %li	Long

Format Specifier	Description
%lf	Double
%lu	Unsigned int or unsigned long
%lli or %lld	Long long
%llu	Unsigned long long
%u	Unsigned int
%p	Pointer
%s	String

Escape Sequence

\t	-	Tab
\n	-	NewLine
\b	-	Backspace
\"	-	Double quotes
\'	-	Single quotes
\\	-	Backslash
\?	-	Question mark
%%	-	Percentage

Input and Output

Reading data from user or external sources

- C provides a standard set of functions.
- These functions are part of the standard input/output library <stdio.h>.
- Most commonly used functions for Input is scanf().

scanf()

- used to read user input from the console
- Syntax : scanf("formatted_string", address_of_variables/values);

Example :

```c
#include <stdio.h>

int main() {
    int age;
    printf("Enter your age: ");
    // Reads an integer
    scanf("%d", &age);
    // Prints the age
    printf("Age is: %d\n", age);
    return 0;
}
```

Problem: Reading char after int

- When taking a char after reading an int, it may not work as expected.
- Reason: scanf() leaves a newline (\n) in the buffer.

What is a Buffer?

- When we give input in C (like typing into the console), it goes through a buffer — a temporary memory area that stores what we type before it gets processed.
- If you type: 5⏎, you're actually typing two characters:
  5 → the digit
  '\n' → the newline character when you press Enter (⏎)

How scanf() works

- scanf() reads the 5 from the buffer and stores it in x. But the newline character \n (from pressing Enter) is left behind in the buffer.
- So the buffer now still contains: \n
- Now, if you write:
  char ch;
  scanf("%c", &ch);
- It immediately reads the \n left in the buffer and stores it in ch. You think you're reading a character (like y), but it reads newline instead!

Solution

1. Use a space before %c to tell scanf() to skip any whitespace, including: Spaces, Tabs, Newlines (\n).

```c
scanf(" %c", &ch);
```

2. Use fflush(stdin): Clears the input buffer to remove any leftover. characters (such as spaces, tabs, or newlines) before reading input.

```c
fflush(stdin);  // Clears the input buffer
scanf("%c", &ch);  // Reads the next character
```

Operators

Operators are symbols used to perform operations on variables and values. The values and variables used with operators are called operands.

Types of Operator

1. Arithmetic Operators :

It divided into two categories -

a. +, -, *, /(int/int will always yield int) , % (Return remainder after dividing two numbers).

Special powers of / & % by powers of 10

/ : to reduce the number by 1 digit

% : to get last digit(s) of number.

b. Increment (++) and Decrement (--) Operators

1. Pre-Increment/Decrement(++a/ --a) : The value is increased/ decreased first, then used in the expression.

```
int a = 5;
int b = ++a;  // a becomes 6, then b =
6
```

2. Post-Increment/Decrement(a++/ a--) : The value is used first, then increased/decreased.

Rules:

1. Can only be used with variables, not constants.
   - ✅ a++;
   - ❌ 5++; → Not allowed
2. Works only with numeric data types (int, float, etc.), not with strings or arrays.

2. Relational Operators

- Used to compare two values. Returns 1 for true, 0 for false.
- <, >, <=, >=, ==, !=
- Equal To (==)
  Checks if two operands are equal.
- Not Equal To (!=)
  Checks if two operands are not equal.
- Greater Than or Equal To (>=)
  Checks if one operand is either greater than or equal to the other.
- Less Than or Equal To (<=)
  Checks if one operand is either less than or equal to the other.

3. Logical Operators

- Used for combining multiple conditions.
- &&, ||, !

Logical AND Operator(&&)

Returns true when both conditions under evaluation are true, otherwise it returns false.

e.g : if(a>b && a);

Logical OR Operator(||)

Returns true if any one of the given conditions is true, otherwise it returns false. It returns false if and only if both conditions under evaluation are false.

e.g : if(a>b || a<c) System.out.println("Max : " + a);

Logical Not Operator(!)

It accepts a single value as an input and returns the inverse of the same. This is a unary operator unlike the AND and OR operators.

e.g : if(!(a<c) System.out.println("Max : " + a);

4. Assignment Operators

- Used to assign and update values.
- +=, -=, *=, /=, %=
- Example : a = a+5, we can write a += 5.

5. sizeof Operators

- Used to get the size in bytes of a data type or variable.
- sizeof(int) // Output: 4 (usually)

Header Files

A header file contains pre-written code (like functions, constants, macros) that we can use in our programs to save time and effort.

We use #include to include a header file at the top of our code.
#include <header_file_name.h>

Common Header Files

- stdio.h - For input/output functions like printf() and scanf()
- stdlib.h - For general functions like memory allocation, etc.
- string.h - For string functions like strlen(), strcpy(), strcmp()
- math.h - For math functions like sqrt(), pow(), ceil(), floor()

Functions from math.h

- sqrt(x) - Square root of x
- pow(x, y) - x raised to the power y
- ceil(x) - Smallest integer ≥ x (round up)
- floor(x) - Largest integer ≤ x (round down)

Control Flow Statements

Programs can make decisions based on conditions. This is called decision making, and the statements used for it are called conditional statements.

Types of Conditional Statements

1. if Statement

- Used to check one condition.
- If the condition is true, the block runs.

Syntax :

```
if (condition) {
    // code runs if condition is true
}
```

Example :

```
if (age > 18) {
    printf("You can vote");
}
```

2. if-else Statement

- Adds an alternative if the condition is false.

Syntax :

```
if (condition) {
    // code if true
} else {
    // code if false
}
```

Example :

```
if (marks >= 40) {
    printf("Pass");
} else {
    printf("Fail");
}
```

3. if-else if-else Ladder

- Used to check multiple conditions one by one.
- As soon as one condition is true, its block runs and the rest are skipped. If none are true, the final else runs.

Syntax :

```
if (condition1) {
    // if condition1 true
} else if (condition2) {
    // if condition2 true
} else {
    // if none true
}
```

Example :

```c
if (marks >= 90) {
    printf("Grade A");
} else if (marks >= 75) {
    printf("Grade B");
} else {
    printf("Grade C");
}
```

4. if Ladder (Multiple Independent ifs)

- All conditions are checked separately, even if one is already true.

Syntax :

```c
if (condition1) {
    // code
}
if (condition2) {
    // code
}
```

Example :

```c
if (num % 2 == 0) {
    printf("Even");
}
if (num % 3 == 0) {
    printf("Divisible by 3");
}
```

Advance Control Flow

Ternary Operator

- Shorthand for if-else
- Syntax : condition ? value_if_true : value_if_false;
  - If the condition is true, the expression after the ? gets executed.
  - If the condition is false, the expression after the : gets executed.

Example :

```c
int age = 20;

// Using ternary operator to decide eligibility
(age >= 18) ? printf("You are eligible to vote.\n") :
printf("You are not eligible to vote.\n");
```

Type Conversion

- Implicit Casting: Done automatically by the compiler.
  (e.g., int to float)
- Explicit Casting: Explicit type conversion (also called type casting) is used when you want to manually convert one data type into another.
  Syntax : (data_type) value;

```c
int a = 5, b = 2;
float result;

result = (float)a / b;  // cast a to float

printf("Result = %.2f", result);
```

Switch Statement

- Switch statement allows executing one block of code from many based on the value of an expression.
- It's a cleaner alternative to multiple if-else conditions.

Syntax :

```
switch (expression) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // default code
}
```

Example :

```c
#include <stdio.h>
int main() {
    int day = 3;
    switch (day) {
        case 1:
            printf("Monday\n");
            break;
        case 2:
            printf("Tuesday\n");
            break;
        case 3:
            printf("Wednesday\n");
            break;
        case 4:
            printf("Thursday\n");
            break;
        case 5:
            printf("Friday\n");
            break;
        default:
            printf("Weekend\n");
    }
    return 0;
}
```

Key Points:

- Works with int and char types only.
- Each case must have a unique value.
- break is optional but prevents fall-through.
- default is optional; runs if no match.
- Nesting of switch is allowed.
- default can be placed anywhere.

Fall-through in switch Statement

- Fall-through occurs in a switch statement when break is not used after a case.
- This causes the control to continue executing the next case(s) even if they don't match the expression until break found.

```c
#include <stdio.h>

int main() {
    int num = 2;

    switch (num) {
        case 1:
            printf("One\n");
        case 2:
            printf("Two\n");
        case 3:
            printf("Three\n");
        default:
            printf("Default\n");
    }

    return 0;
}
```

Output :
Two
Three
Default

Using Range in switch Case

You can use a range of numbers instead of a single number or character in the case statement.
That is the case range extension of the GNU C compiler and not standard C.

Syntax
case low ... high:

- Matches any value between low and high (inclusive).
- Only works in GCC.

Example:

```c
#include <stdio.h>

int main() {
    int arr[] = { 1, 5, 15, 20 };

    for (int i = 0; i < 4; i++) {
        switch (arr[i]) {
            case 1 ... 6:
                printf("%d in range 1 to 6\n", arr[i]);
                break;
            case 19 ... 20:
                printf("%d in range 19 to 20\n", arr[i]);
                break;
            default:
                printf("%d not in range\n", arr[i]);
                break;
        }
    }
    return 0;
}
```

Output:
1 in range 1 to 6
5 in range 1 to 6
15 not in range
20 in range 19 to 20

Loops

Why Loops are Necessary

- Sometimes we need to repeat the same code multiple times.
- Using printf() repeatedly (e.g., 100 or 1000 times) is inefficient.
- Loops help reduce redundancy and improve code readability.

What are Loops?

- Loops allow repeating a block of code until a condition is met.
- Saves time and avoids rewriting code.
- Used for automation and iteration in programs.

Types of Loops

1. **Entry-Controlled Loops:**
   Condition is checked before executing the loop body.
   - for loop
   - while loop
2. **Exit-Controlled Loop:**
   Condition is checked after executing the loop body.
   - do-while loop

for Loop

- Repeats a block of code a known number of times.

Syntax :

```
for (initialization; condition; updation) {
    // loop body
}
```

Example :

```c
for (int i = 0; i < 5; i++) {
    printf("%d ", i);
}
```

Infinite Loops

- An infinite loop is a loop that never terminates on its own.
- The condition is always true or there is no condition at all.

Example :

```c
for(;;) {
    // runs forever
}
```

```c
for(;;);
```

Since no condition is specified, it never becomes false.

Break Statement

- Exits from the loop immediately.

Example :

```c
for (int i = 0; i < 10; i++) {
    if (i == 5)
        break;
}
```

Output :
The loop runs 5 times but shows no output as there's no print statement. It exits on the 6th iteration due to break.

Continue

- Skips current iteration and moves to the next.

Example :

```c
for (int i = 0; i < 5; i++) {
    if (i == 2)
        continue;
    printf("%d ", i);
}
```

Output :
0 1 3 4
The loop prints all values except 2 because continue skips the iteration when i == 2.

Flowcharts

Flowchart

- A flowchart is a diagrammatic representation of an algorithm or logic.
- It shows the step-by-step flow of solving a problem.
- Used before coding to plan and visualize the logic clearly.

Why Use Flowcharts

- Helps in understanding and organizing logic before coding.
- Useful for debugging and finding logical errors early.
- Makes it easier to explain your approach to others, especially in group projects.
- Acts as a visual documentation of your program.
- Gives a clear idea of the execution path.

Flowchart Symbols

- **Terminators**

Used to denote the start point of the program.

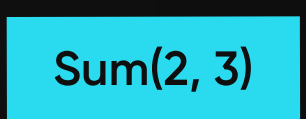
Used to denote the end point of the program.

- **Input/Output**

Used for taking input from the user and store it in variable 'n'.

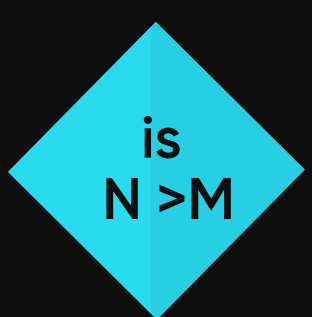
Used to output value stored in variable 'n'.

- **Processing**



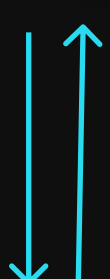
Used to perform the operation(s) in the program. For example: Sum(2, 3) just performs arithmetic summation of the numbers 2 and 3.

- **Condition**



Used to make decision(s) in the program means it depends on some condition & answers in the form of TRUE(for yes) and FALSE(for no).

- **Arrows**

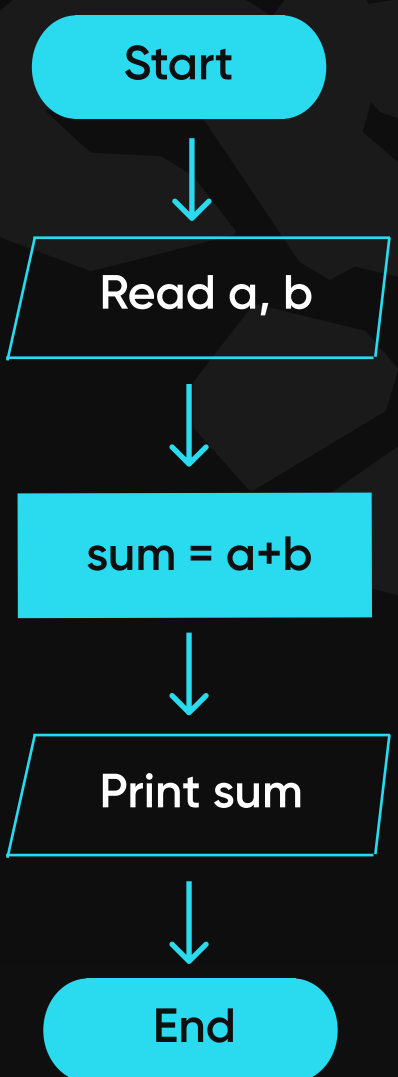


Generally, used to show the flow of the program from one step to another. The head of the arrow shows the next step and the tail shows the previous one.

- **Connector**

Used to connect different parts of the program and are used in case of break-through. Generally, used for functions(which we will study in our further sections).

- **Example : Flowchart of Adding Two Numbers a & b**



while Loop

while Loop

- The while loop executes repeatedly as long as the condition is true.
- It is suitable when the number of iterations is not known in advance.

Syntax :

```
while (condition) {
    // code block to be executed
}
```

Example :

```
int i = 1;
while (i <= 5) {
  printf("Count: %d ", i);
  i++;
}
```

Output :
Count: 1 Count: 2 Count: 3 Count: 4 Count: 5

Infinite while Loop

- loop runs forever unless you use break, an external condition, or terminate the program manually.

Example :

```
while(1) {
    // runs infinitely
}
```

do-while Loop

do-while Loop

- Executes the loop at least once, regardless of whether the condition is true or false
- Useful when you want the loop body to run at least one time, such as showing a menu before checking input

Syntax :

```
do {
    // body of loop
} while(condition);
```

Example :

```
int i = 1;
do {
    printf("%d ", i);
    i++;
} while(i <= 5);
```

Output :
1 2 3 4 5

Nested Loops

Nested Loop

- A nested loop means having one loop inside another loop.
- We can define any number of loops inside another loop.

Syntax :

```
for ( initialization; condition; increment ) {
   for ( initialization; condition; increment ) {
      // statement of inside loop
   }
   // statement of outer loop
}
```

Example :

```
// Outer loop for rows
for (int i = 0; i < n; i++) {
   // Inner loop for columns
   for (int j = 0; j < n; j++) {
       printf("* ");
   }
   printf("\n"); // Move to next row
}
```

Output :

```
* * * *
* * * *
* * * *
* * * *
```

Functions

Functions

- A function is a set of statements that perform a specific task when called.
- It promotes modularity and code reusability.

Syntax of Functions

The syntax of functions can be divided into three main parts:

1. Function Declaration (Prototype)
2. Function Definition
3. Function Call

1. Function Declaration (Prototype)

The syntax of functions can be divided into three main parts:

- Declares the function before its use.
- Tells the compiler about the function name, return type, and parameters.
- Syntax: return_type function_name(parameter_list);

2. Function Definition

- Contains the actual body (code) of the function.
- Syntax :

```
return_type function_name(parameter_list) {
  // statements
}
```

3. Function Call

- Executes the function using its name and arguments.
- Syntax:

```
function_name(arguments);
```

Return Types

- Specifies what type of value the function will return.
- If a function doesn't return anything, we use void as return type.

Example :

```
void greet();  // returns nothing
int add();     // returns an int
```

Parameters (Arguments)

- Inputs passed to a function for processing.
- Declared in the function definition and prototype.
- Can have zero or more parameters.

Example :

```
void printName(char name[]);  // takes one parameter
int sum(int a, int b);        // takes two parameters
```

Passing Parameters to Functions

- The values/variables passed to a function during the function call are actual parameters.
- The variables used in the function definition to receive the values are formal parameters.

Example :

```c
#include <stdio.h>

void addTen(int x) { //Formal Parameters
    x = x + 10;
    printf(" x = %d\n", x);
}

int main() {
    int num = 5;
    addTen(num); //Actual Parameters
    printf("Outside function: num = %d\n", num);
    return 0;
}
```

Ways to pass parameters

We can pass arguments to the function in two ways:

1. Pass by Value
2. Pass by Reference

1. Pass by Value

- A copy of the actual parameter is passed to the function.
- Changes made inside the function do not affect the original variable.
- Value inside main function remains same.

Example :

```c
#include <stdio.h>

void swap(int var1, int var2){
  int temp = var1;
  var1 = var2;
  var2 = temp;
}
int main(){
  int var1 = 3, var2 = 2;
  printf("Before swap var1 and var2 : %d, %d\n",
          var1, var2);
  swap(var1, var2);
  printf("After swap var1 and var2 : %d, %d",
          var1, var2);
  return 0;
}
```

Output :
Before swap var1 and var2 : 3, 2
After swap var1 and var2 : 3, 2

2. Pass by Reference (DTL)

- In Pass by Reference, the memory address of the variable is passed to the function.
- Changes made to the parameter inside the function directly affect the original variable.

Recursion

What is Recursion?

- Recursion = a function calling itself.
- Used when a problem can be broken into smaller identical subproblems.
- Continues until a base case stops it.

Core Components of Recursion

1. Base Case

- Condition where recursion stops
- Prevents infinite calls
- Smallest possible input

```
if(n == 0) return 1;
```

2. Recursive Case

- Function calls itself with smaller input.

```
return n * factorial(n-1);
```

3. How Recursion Works (Call Stack)

- Each function call goes into stack memory
- Execution pauses until inner calls return

```
main() → f(3)
      → f(2)
        → f(1)
          → f(0) ← base case
```

- Then returns back:

```
f(0) → f(1) → f(2) → f(3)
```

4. Example: Factorial Program

```c
#include <stdio.h>

int factorial(int n) {
    if(n == 0)  // base case
        return 1;
    return n * factorial(n - 1); // recursive case
}

int main() {
    printf("%d", factorial(5));
    return 0;
}
```

5. Step-by-Step Execution (factorial(3))

```
factorial(3)
= 3 * factorial(2)
= 3 * 2 * factorial(1)
= 3 * 2 * 1 * factorial(0)
= 3 * 2 * 1 * 1
= 6
```

Arrays

Why?

- Suppose you want to store the names of 1000 people.
- One way is to create 1000 different variables but this is not only time-consuming and inefficient, but also difficult to manage.
- At that time we use an array that can store multiple values under a single variable name.

What is Array?

- An array is a fixed-size collection of similar data items.
- All elements in an array are stored in contiguous memory locations.
- It used to store a group of values under a single variable name.

Declaration of Array

Syntax :

```
data_type array_name [size];
```

Example :

```
int arr[5];
```

Initialization

- Array Initialization with Declaration

```
data_type array_name[] = {1,2,3,4,5};
```

- Array Initialization with Declaration without Size

```
data_type array_name [size] = {value1, value2, ... valueN};
```

Array Addressing(Contiguous Memory Allocation)

- Contiguous Memory: Array elements are stored in sequential (contiguous) memory locations.
- Hexadecimal Addressing: Internally, memory addresses are in hexadecimal format (e.g., 0x100, 0x104, 0x108, etc.).
- Base Address: The memory address of the first element in the array is called the base address.
- Access by Index: Array elements are accessed using index numbers, starting from 0.
- Address Calculation Formula:

Address of arr[i] = Base address + (i * size of data type)

- Example: If Base Address = 100 and size of int = 4 bytes, then
  - arr[0] → 100
  - arr[1] → 104
  - arr[2] → 108
  - And so on...

2D Array

Arrays are used to store multiple values of the same data type in a single variable.
Based on the number of dimensions, arrays can be classified into two types:

1. One Dimensional Array (1D Array)

- These arrays have only one row of elements, like a simple list.
- Used when you want to store a sequence of items, like marks of students, prices, etc.

2. Multidimensional Array

- Multidimensional arrays are arrays with more than one dimension.
- Used when data needs to be stored in tabular or grid-like format. Common types:
  - 2D Arrays – Tables, matrices.
  - 3D Arrays – Cubes or multi-layered data.
- Arrays beyond 3D are possible but rarely used due to complexity and memory usage.

2D Array

- A 2D array is essentially an array of arrays.
- It can be visualized like a table with rows and columns.

Declaration

```
data_type array_name[size1][size2];
```

Initialization

- int matrix[2][3] = {
  {1, 2, 3},
  {4, 5, 6}
  };

- int matrix[2][3] = {1, 2, 3, 4, 5, 6}; // Same result

Accessing Elements

- You can access elements using two indices:
- printf("%d", matrix[1][2]); // Output: 6 (2nd row, 3rd column)

Traversing a 2D Array

```c
#include <stdio.h>

int main(){
  // declaring and initializing 2d array
  int arr[2][3] = { 10, 20, 30, 40, 50, 60 };

  printf("2D Array:\n");
    // printing 2d array
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ",arr[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output :
1 2 3
4 5 6

Strings

A String is a sequence of characters terminated with a null character '\0'.

char str[] = {'s', 'h', 'e', 'r', 'y', '\0'};

Declaration

- Declare as a one-dimensional array.

```
char string_name[size];
```

String Intialization

1. Assigning a String Literal without Size

```
char str[] = "GeeksforGeeks";
```

2. Assigning a String Literal with a Predefined Size

```
char str[50] = "GeeksforGeeks";
```

3. Assigning Character by Character with Size

```
char str[14] = {
'G','e','e','k','s','f','o','r','G','e','e',
'k','s','\0'};
```

4. Assigning Character by Character without Size

```
char str[] = {
'G','e','e','k','s','f','o','r','G','e','e',
'k','s','\0'};
```

NOTE :

- In C, when a string is enclosed in double quotes (" "), the compiler automatically adds a null character '\0' at the end.
- This '\0' marks the end of the string and is essential for string operations.

Read String Input

1. Using scanf()

    - Used to read single-word strings (stops at whitespace).
    - Example:

```
char name[50];
scanf("%s", name);
```

2. Using gets() (Deprecated)

    - Reads a full line including spaces until newline (\n) is encountered.
    - Deprecated due to buffer overflow issues (unsafe).

- Example

```c
char name[50];
gets(name);  // Not recommended
```

3. Using fgets() (Recommended)

- Safely reads a string including spaces.
- Reads until newline or given size - 1.
- Automatically appends null character (\0).
- Syntax : fgets(name, size, stdin);
- Example

```c
char name[50];
fgets(name, 50, stdin);
```

4. Using scanf() with Scanset

- Reads a full line with spaces until a newline is encountered.
- %[^\n]s: Tells scanf to read everything until a newline (\n).
- Example

```c
char str[100];
scanf("%[^\n]s", str);
```

Common String Functions

- You need to include the header: #include <string.h>
- Some functions are :

- strlen(str) : Returns the length of the string string(excluding the null character \0).
- strcpy(dest, src) : Copies the content of one string into another.
- strcmp(str1, str2) Compares two strings. Return 0 if both strings are equal, +ve if str1 < str2, -ve if str1 > str2 (based on ASCII).
- strcat(dest, src) Concatenates (joins) two strings.

Example

```c
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Hello";
    char dest[20];
    strcpy(dest, src);  // dest becomes "Hello"
    int len = strlen(src);  //5
    strcmp("apple", "apple");  // Returns 0
    strcmp("apple", "ball");   // Returns -ve < 0

    char a[20] = "Hello ";
    char b[] = "World";
    strcat(a, b);  // a becomes "Hello World"
    return 0;
}
```

Pointers

- A pointer is a variable that stores the memory address of another variable.
- It doesn't hold the actual value, but the address of where that value is stored.
- This allows us to manipulate the data stored at a specific memory location without actually using its variable.

Declaration

```
data_type* pointer_name;
```

- data_type specifies the type of variable the pointer will point to.
- The * symbol indicates that the variable is a pointer.

Example :

```
int *ptr;      // pointer to an integer
float *fptr;   // pointer to a float
char *cptr;    // pointer to a character
```

Initializing a Pointer

A pointer is initialized by assigning it the address of a variable using the & (address-of) operator.

```
int a = 10;
int *ptr = &a;   // ptr stores the address of variable a
```

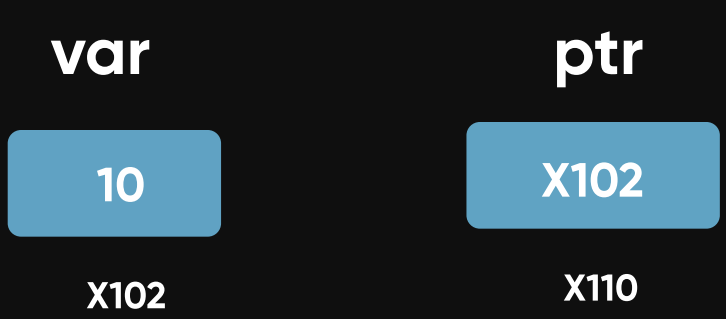
- Now, ptr points to a and can be used to access or modify the value of a indirectly.

Dereference a Pointer

- Accessing the value stored at the memory address(pointer points to).

Example

```c
#include <stdio.h>

int main() {
    int var = 10;

    // Pointer storing the address of var
    int* ptr = &var;

    // Direct access (prints address)
    printf("%p\n", ptr);

    // Dereferencing to get value (prints 10)
    printf("%d\n", *ptr);

    return 0;
}
```



- If you print *ptr(value it ptr) it will print 10.

Passing Pointer to Function

- In this method, we pass the address (memory location) of variables to the function.
- This is also called Pass by Reference.
- Changes made inside the function are reflected in the original variables, since both point to the same memory location.

Why use this?

- Allows functions to modify actual variables.
- Useful for swapping, modifying arrays, and for performance in large data structures.
- Also saves memory and time by avoiding data copying.

Example: Swapping Two Values Using Pointers

```c
#include <stdio.h>
// Function to swap two values using pointers
void swap(int* a, int* b){
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main(){
    int x = 10, y = 20;

    // Passing addresses of x and y
    swap(&x, &y);
    printf("Values after swap a:%d, b:%d", x, y);
    return 0;
}
```

Pointer to Pointer(Double Pointer)

- A pointer to pointer (or double pointer) is a variable that stores the address of another pointer variable.

Declaration

- int **ptr; means ptr is a pointer to a pointer to an integer.

```
data_type **pointer_name;
```

Example

```c
#include <stdio.h>

int main() {
    int num = 100;

    // Single pointer
    int *ptr = &num;

    // Double pointer
    int **dptr = &ptr;

    // Printing values
    printf("Value of num: %d\n", num);
    printf("Value using *ptr: %d\n", *ptr);
    printf("Value using **dptr: %d\n", **dptr);

    return 0;
}
```

Dynamic Memory Allocation

- Arrays have fixed sizes defined at compile time.
- Problems with static allocation:
  a. May not fit all required data.
  b. May waste memory if oversized.
- Dynamic Memory Allocation (DMA) lets you:
  - Allocate memory at runtime.
  - Resize memory as needed.
  - Optimize memory usage.

Functions for Dynamic Memory Allocation

These are provided by the <stdlib.h> header:

- malloc() Allocates memory block (uninitialized)
- calloc() Allocates memory block (initialized to 0)
- free() Frees allocated memory
- realloc() Resizes the previously allocated memory

malloc() – Memory Allocation

- Allocates a single block of memory of given size (in bytes).
- Returns a void pointer to the first byte of the block.
- Memory is uninitialized (contains garbage values).
- Returns NULL if memory allocation fails.

Syntax

```
ptr = (type *) malloc(size_in_bytes);
```

Example

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocating memory for 5 integers (5 * 4 bytes = 20 bytes)
    int *ptr = (int *)malloc(5 * sizeof(int));

    // Check if memory allocation is successful
    if (ptr == NULL) {
        printf("Memory allocation failed!");
        return 1;
    }

    // Assign and print values
    for (int i = 0; i < 5; i++) {
        ptr[i] = i + 1;
        printf("%d ", ptr[i]);
    }
    return 0;
}
```

Output :

1 2 3 4 5

calloc() – Contiguous Allocation

- calloc() stands for contiguous allocation.
- Similar to malloc(), but with one key difference:
  - Initializes all allocated memory to zero.

Syntax

```c
ptr = (type *) calloc(number_of_elements, size_of_each_element);
```

Example

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocating memory for 5 integers and initializing them to 0
    int *ptr = (int *)calloc(5, sizeof(int));

    // Check if memory allocation is successful
    if (ptr == NULL) {
        printf("Allocation Failed");
        exit(0);
    }

    // Print values (all will be 0)
    for (int i = 0; i < 5; i++) {
        printf("%d ", ptr[i]);
    }
    return 0;
}
```

Output :
0 0 0 0 0

free()

- Memory allocated using malloc() or calloc() is not freed automatically.
- free() is used to release dynamically allocated memory back to the OS.
- Helps to prevent memory leaks.

Syntax

```c
free(ptr);
```

Example

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)calloc(5, sizeof(int));
    // Print values
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    // Free the memory
    free(ptr);
    ptr = NULL;

    return 0;
}
```

- After free(), the pointer becomes invalid.
- Always set the pointer to NULL after freeing:

realloc() – Resize Allocated Memory

- realloc() is used to resize a previously allocated memory block.
- Can expand or shrink the size of the memory.
- Keeps existing data intact up to the new size.

Syntax

```c
realloc(ptr, new_size);
```

- ptr: Pointer to previously allocated memory.
- new_size: New size in bytes.
- Returns a new pointer to the memory or NULL on failure.

Example

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocate memory for 5 integers
    int *ptr = (int *)calloc(5, sizeof(int));
    // Resize memory to hold 10 integers
    ptr = (int *)realloc(ptr, 10 * sizeof(int));
    // Check if reallocation was successful
    if (ptr == NULL) {
        printf("Memory Reallocation Failed");
        exit(0);
    }
    // Use the reallocated memory
    for (int i = 0; i < 10; i++)
        ptr[i] = i + 1;

    for (int i = 0; i < 10; i++)
        printf("%d ", ptr[i]);
    // Free memory
    free(ptr);
    ptr = NULL;

    return 0;
}
```

Output : 1 2 3 4 5 6 7 8 9 10

Structure

- A structure is a user-defined data type.
- It allows grouping variables of different data types into a single unit.
- Useful for representing real-world entities like student records, employee info, etc.
- Defined using the struct keyword.

Syntax : Structure Definition

```
struct StructureName {
    data_type member1;
    data_type member2;
    ...
};
```

Creating Structure Variable

Method 1: Declare Variables After the Structure Definition

```
struct Student {
    char name[50];
    int age;
    float grade;
};

// Declaring structure variable
struct Student s1, s2;
```

Method 2: Declare Variables With Structure Definition

```c
struct Student {
    char name[50];
    int age;
    float grade;
} s1, s2;
```

Accessing Structure Members

1. Using Dot Operator (.)
   - When you have a structure variable, use the dot (.) operator to access or modify its members.

```c
structure_variable.member1;
structure_variable.member2;
```

2. Using Arrow Operator (->)
   - When you have a pointer to a structure, use the arrow (->) operator to access members.

```c
structure_pointer->member1;
structure_pointer->member2;
```

Example : for structure variable

```c
struct Student {
    int roll;
    char name[50];
};

struct Student s1;
s1.roll = 101;
strcpy(s1.name, "Shery");
```

Example : for pointer to a structure

```c
struct Student s1 = {101, "Shery"};
struct Student *ptr = &s1;

printf("%d\n", ptr->roll);
printf("%s\n", ptr->name);
```

Initialize Structure Members

1. Default Initialization
    - Structure members are not automatically initialized.
    - Without initialization, they may contain garbage values.

```c
structure_variable.member1;
structure_variable.member2;
```

Example :

```c
struct Point {
    int x, y;
};

struct Point p = {0};  // Both x and y = 0
```

2. Initialization Using Assignment

Example :

```c
struct Point p;
p.x = 5;
p.y = 10;
```

Copying Structures

- Structure variables in C can be copied directly using the assignment operator:
- Syntax : s2 = s1

```c
struct Student {
    int id;
    char grade;
};
int main() {
    struct Student s1 = {1, 'A'};
    struct Student s2;
    s2 = s1;  // Direct copy
    printf("s2.id = %d, s2.grade = %c\n", s2.id, s2.grade);
    return 0;
}
```

- If the structure contains pointers, only the pointer addresses are copied, not the data they point to.

Passing Structures to Functions

- **Pass by Value** – makes a copy
- **Pass by Pointer** – avoids copying, can modify original

```c
#include <stdio.h>

struct A {
    int x;
};
// a is passed by value, b is passed by pointer
void increment(struct A a, struct A* b) {
    a.x++;      // Only local copy of a is changed
    b->x++;     // Actual b is modified
}

int main() {
    struct A a = {10};
    struct A b = {10};

    increment(a, &b);

    printf("a.x: %d \tb.x: %d\n", a.x, b.x);
    return 0;
}
```

Output : a.x: 10 b.x: 11

- a was passed by value, so a.x++ only affected the local copy.
- b was passed by pointer, so b->x++ updated the actual variable.

Size of Structures

- In theory, the size of a structure should be the sum of the sizes of its members.
- **But in reality,** Most compilers add padding between members to align data in memory for faster access.

What is Structure Padding?

- Structure Padding is the addition of unused memory bytes between members to ensure proper alignment of data members in memory.

Example : with padding

```c
#include <stdio.h>

struct A {
    char c;    // 1 byte
    int i;     // 4 bytes (aligned to 4 bytes)
};

int main() {
    printf("Size of struct A: %lu\n", sizeof(struct A));
    return 0;
}
```

Output (usually) : Size of struct A: 8
Explanation:

- char c uses 1 byte
- 3 bytes padding are added for alignment
- int i uses 4 bytes
- Total = 1 + 3 + 4 = 8 bytes

What if you want to disable padding?

- Use Structure Packing to tightly pack members without extra padding.

```c
#include <stdio.h>
#pragma pack(1) // Disable padding
struct B {
    char c;
    int i;
};
int main() {
    printf("Size of struct B: %lu\n", sizeof(struct B));
    return 0;
}
```

Output : Size of struct B: 5

Array of Structures

- An array of structures is a collection of structure variables of the same type stored in contiguous memory locations.

```c
#include <stdio.h>

struct Student {
    char name[20];
    int age;
    float marks;
};
int main() {
    // Declare an array of 3 Students
    struct Student students[3] = {
        {"Alice", 18, 85.5},
        {"Bob", 19, 90.0},
        {"Charlie", 17, 92.5}
    };

    for (int i = 0; i < 3; i++) {
        printf("Name: %s, Age: %d, Marks: %.2f\n",
               students[i].name, students[i].age, students[i].marks);
    }
    return 0;
}
```

typedef

- typedef is a keyword in C that allows you to create a new name (alias) for an existing data type.
- This helps make code more readable, manageable, and easier to work with — especially for complex or user-defined types.

Example :

```c
#include <stdio.h>

typedef int Integer;

int main() {
    Integer n = 10;  // Integer is now an alias for int
    printf("%d", n);
    return 0;
}
```

typeof with Structures:

- Without typedef:

```c
struct Student {
    int id;
    char name[50];
};

struct Student s1;
```

- With typedef:

```c
typedef struct {
    int id;
    char name[50];
} Student;

Student s1;  // Cleaner syntax
```

Nested Structures

- One structure inside another.
- This helps you create more organized and complex data types, especially when grouping related data.

Types of Nesting

1. Separate Structure Nesting
- Both inner and outer structures are declared separately, and one is used inside the other.

2. Embedded Structure Nesting
- The inner structure is declared inside the outer (parent) structure.

How to Access Nested Members?

- Use the dot (.) operator twice:

```c
str_parent.str_child.member;
```

- Example: Separate Structure Nesting

```c
#include <stdio.h>
// Child structure declaration
struct child {
    int a;
    char c;
};
// Parent structure declaration
struct parent {
    int b;
    struct child d;
};
int main() {
    struct parent p = { 100, {200, 'L'} };
    // Accessing and printing nested members
    printf("p.b = %d\n", p.b);
    printf("p.d.a = %d\n", p.d.a);
    printf("p.d.c = %c", p.d.c);
    return 0;
}
```

- Example : Embedded Structure Nesting

```c
#include <stdio.h>

struct parent {
    int a;
    // Inner structure declared inside parent
    struct {
        int x;
        char c;
    } b;
};
int main() {
    struct parent p = { 100, {200, 'L'} };
    printf("p.a = %d\n", p.a);
    printf("p.b.x = %d\n", p.b.x);
    printf("p.b.c = %c", p.b.c);

    return 0;
}
```

Union

- A union is also a user-defined data type.
- Unlike structure, all union members share the same memory location, so only one member can contain a valid value at a time.
- Useful to save memory when you only need to use one value at a time

Syntax : Union Declaration

```
union UnionName {
    type1 member1;
    type2 member2;
    ...
};
```

Variable Creation

Method 1: With Declaration

```
union UnionName {
    int a;
    float b;
} u1, u2;
```

Method 2: After Declaration

```
union UnionName {
    int a;
    float b;
} u1, u2;
```

Access Union Members

- Use the dot (.) operator, just like structures:

```
u1.a = 5;
printf("%d", u1.a);
```

Example

```
#include <stdio.h>
#include <string.h>
union A {
    int i;
    float f;
    char s[20];
};
int main() {
    union A a;

    // Storing an integer
    a.i = 10;
    printf("data.i = %d\n", a.i);
    // Storing a float (overwrites integer)
    a.f = 22.5;
    printf("data.f = %.2f\n", a.f);
    // Storing a string (overwrites float)
    strcpy(a.s, "GfG");
    printf("data.s = %s\n", a.s);
    return 0;
}
```

Size of Union

- The size of the union will always be equal to the size of the largest member of the union.
- Even though the union contains int, float, and char[20], only the size of the largest member (in this case, char s[20]).

Example :

```c
#include <stdio.h>

union A{
    int x;
    char y;
};
union B{
    int arr[10];
    char y;
};
int main() {
    printf("Sizeof A: %ld\n", sizeof(union A));
    printf("Sizeof B: %ld\n", sizeof(union B));
    return 0;
}
```

Output :
Sizeof A: 4
Sizeof B: 40

#define

- #define is a preprocessor directive used to define:
  - Constants
  - Macros (function-like expressions)
  - Generic macros (with _Generic keyword)

Syntax : Define a Constant

```
#define MACRO_NAME value
```

Syntax : Define an Expression

```
#define MACRO_NAME (expression)
```

Syntax : Define a Macro with Parameters (Function-like Macro)

```
#define MACRO_NAME(arg1, arg2, ...) (expression using args)
```

Example : Define a Constant

```c
#include <stdio.h>
#define PI 3.14159265359

int main() {
    int radius = 21;
    int area = PI * radius * radius;
    printf("Area of Circle of radius %d: %d", radius, area);
    return 0;
}
```

Output : Area of Circle of radius 21: 1385

Example : Define an Expression

```c
#include <stdio.h>
#define PI (22 / 7)

int main() {
    int radius = 7;
    int area = PI * radius * radius;
    printf("Area of Circle of radius %d: %d", radius, area);
    return 0;
}
```

Output : Area of Circle of radius 7: 147

Example : Function-like Macros

```c
#include <stdio.h>
#define CIRCLE_AREA(r) (3.14 * r * r)
#define SQUARE_AREA(s) (s * s)

int main() {
    int radius = 21, side = 5, area;
    area = CIRCLE_AREA(radius);
    printf("Area of Circle of radius %d: %d\n", radius, area);
    area = SQUARE_AREA(side);
    printf("Area of square of side %d: %d", side, area);
    return 0;
}
```

Output : Area of Circle of radius 7: 147

Important Facts :

- Do not use = in #define. Example: #define PI 3.14 is correct, #define PI = 3.14 is wrong.
- Do not end #define statements with a ;.
- You cannot assign variables to macros.
- Values defined with #define cannot be changed at runtime.
- Macros are replaced during preprocessing, before actual compilation starts.

enumeration

- An enum (short for enumeration) is a user-defined data type in C. It assigns names to a set of integer constants, making the code:
  - Easier to read
  - Easier to maintain
  - More meaningful than just using raw numbers

Syntax : Define a Constant

```
enum enum_name {
    constant1, constant2, constant3, ...
};
```

By default:

- constant1 = 0
- constant2 = 1
- constant3 = 2, and so on...

Example 1: Simple Enum

```
#include <stdio.h>

enum calculate {
    SUM, DIFFERENCE, PRODUCT, QUOTIENT
};

int main() {
    enum calculate op = PRODUCT;
    printf("%d\n", op); // Output: 2
    return 0;
}
```

Creating Enum Variables

- After defining an enum, we create variables like this:

```
enum calculate c1;
```

- Enum with Custom Values

```
enum enm {
    a = 3, b = 2, c
};
```

Then a = 3, b = 2, and c = 3 (incremented from b).

Size of Enum

Enums typically take the size of an int (4 bytes), but can vary by compiler.

```
#include <stdio.h>

enum direction { EAST, NORTH, WEST, SOUTH };

int main() {
    enum direction dir = NORTH;
    printf("%ld bytes", sizeof(dir));  // Likely 4 bytes
    return 0;
}
```

Output : 4 bytes

Enum + Typedef

Use typedef to avoid writing enum repeatedly.

Example

```c
#include <stdio.h>

typedef enum direction {
    EAST, NORTH, WEST, SOUTH
} Dirctn;

int main() {
    Dirctn d = NORTH;
    printf("%d", d);  // Output: 1
    return 0;
}
```

- Enums improve readability and code clarity
- Default values start from 0, or you can assign your own
- Enum variables are usually stored as integers
- Use typedef for convenience

File Handling

Why use file handling ?

- To store program output permanently.
- Helps in reading/writing large data.
- Makes data reusable.

Types of Files

1. Text File (.txt)
    - Stores ASCII characters.
    - Human-readable.
2. Binary File (.bin)
    - Stores data in binary (0s & 1s).
    - Faster, compact, not human-readable.

Basic File Operations

Operation		Function Used
Create/Open	-	fopen()
Read	-	fscanf(), fgets(), fgetc()
Write	-	fprintf(), fputs(), fputc()
Move Pointer	-	fseek(), rewind()
Close File	-	fclose()

File Pointer

- A file pointer is a reference to a particular position in the opened file.
- Declared using the FILE macro.
- The FILE macro is defined in the <stdio.h> header file.

Syntax :

```
FILE *fptr;
```

Create/Open a File

```
FILE *fptr = fopen("fileName", "mode");
```

- fileName: name of the file when present in the same directory as the source file. Otherwise, full path.
- mode: Specifies for what operation the file is being opened.
- If the file is opened successfully, returns a file pointer to it. Otherwise returns NULL.

Modes

- "r" – Read
- "w" – Write (overwrite or create)
- "a" – Append
- "r+", "w+", "a+" – Read + Write
- Add "b" for binary files: "rb", "wb" etc.

Example :

```
#include <stdio.h>
int main() {
    FILE *fptr = fopen("demo.txt", "w");
    if (fptr != NULL) {
        printf("File is created Successfully.!");
    }
    return 0;
}
```

Read a File

- File reading in C is done using functions like fscanf(), fgets(), fgetc(), etc.
- These functions are similar to scanf, gets, etc., but require a file pointer.

Functions		Description
fscanf()	-	Reads formatted input from a file
fgets()	-	Reads a line (string) from a file
fgetc()	-	Reads a single character from a file
fgetw()	-	Reads an integer from a file
fread()	-	Reads bytes from a binary file

- Use fopen() in "r" (read) mode to open the file.
- File pointer moves automatically after each read.
- Reading functions return EOF (End Of File) when the file ends.
- EOF is a constant that tells reading is complete.

Example :

```c
FILE *fptr;
fptr = fopen("fileName.txt", "r");
fscanf(fptr, "%s %s %s %d", str1, str2, str3, &year);
// Reading 100 characters from file
fread(fptr, sizeof(char), 100, fptr);
// Reading a character
char c = fgetc(fptr);
// Reading a line
char line[100];
fgets(line, sizeof(line), fptr);
```

Write to a File

- Use functions like fprintf(), fputs(), fputc() to write data to files.
- These are similar to printf(), puts(), putchar() but used with a file pointer.

Functions		Description
fprintf()	-	Writes formatted text to a file (like printf)
fputs()	-	Writes a string (line) to the file
fputc()	-	Writes a single character to the file
fputw()	-	Writes an integer/number to the file
fwrite()	-	Writes bytes of data to a binary file

- Use fopen() with "w" mode to write (creates a new file or overwrites).
- Use "a" mode for append (adds to existing file).
- File pointer is needed for all write operations.

Example :

```c
FILE *fptr;
fptr = fopen("fileName.txt", "w");
// Writing formatted text
fprintf(fptr, "%s %s %s %d", "We", "are", "in", 2012);
// Writing a single character
fputc('a', fptr);
// Writing a string
fputs("Hello World", fptr);
// Writing a string with size
fwrite("Hello World", sizeof(char), 11, fptr);
```

Closing a File

- fclose() is used to close an opened file.
- It frees the memory and saves changes made to the file.
- Always close a file after reading or writing to avoid data loss.

Syntax :

```
fclose(file_pointer);
```

- file_pointer is the pointer returned by fopen().

Example :

```
FILE *fptr;
fptr = fopen("fileName.txt", "w");

// -------- File operations --------

fclose(fptr); // Closing the file
```

fseek()

- fseek() is used to move the file pointer to a specific location in the file.
- It helps you avoid reading unnecessary records and makes file access more efficient.

Syntax :

```c
int fseek(FILE *ptr, long int offset, int pos);
```

- ptr: File pointer.
- offset: Number of bytes to move the pointer.
- pos: Position from where to start (can be SEEK_SET, SEEK_CUR, or SEEK_END).

- Parameters of fseek():
  a. SEEK_SET: Moves the pointer to a specific position from the beginning of the file.
  b. SEEK_CUR: Moves the pointer relative to the current position.
  c. SEEK_END: Moves the pointer relative to the end of the file.

Example :

```c
// Move pointer to the 10th byte from the beginning of the file
fseek(fp, 10, SEEK_SET);
// Read the next 10 characters from the current position
fgets(buffer, 11, fp);// 11 because we want to read 10 characters + null terminator
// Print the content read from the file
printf("Data from position 10: %s\n", buffer);
```

Explanation:

1. Opening the file: We open the file test.txt in read mode.
2. Using fseek(): The pointer is moved to the 10th byte from the beginning of the file (SEEK_SET).
3. Reading from the file: The fgets() function reads 10 characters from the current position (after seeking).
4. Printing the content: The content read is printed to the console.
5. Closing the file: It's important to close the file after the operation.

Output Example:

1. If the file test.txt has content like:
   Hello, this is a test file.
1. The output will be:
   Data from position 10: is a test f
1. In this case, the file pointer starts at the 10th byte and reads the next 10 characters.